

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a70e4d04a4790e78c.jsonl`  
- SHA-256: `94fc78bec14794f67056eeea1798609011a7923a6988200bda60996d3d3e82c9`  
- Source modified: `2026-07-02T03:00:39+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T02:58:05.138Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.

A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer:

```
{  
  "title": "Default CORS allow_origins=['*'] lets any website script the localhost appliance (paid Gemini runs + local data read)",  
  "category": "security",  
  "severity": "P1",  
  "file": "backend/main.py",  
  "line": 116,  
  "evidence": "backend/main.py:114-116 `_cors_origins_from_env` returns `origins or [\"*\"]`; main.py:123-129 adds CORSMiddleware with allow_methods=['*'], allow_headers=['*']. The TrustedHost guard (main.py:132-156) only blocks DNS-rebinding ? a direct fetch to http://127.0.0.1:<port> from a hostile page sends Host: 127.0.0.1 (allowed) and the wildcard CORS answers its preflight, so evil.com JS can POST /api/process (no auth by default: backend/api/auth.py:126-127 returns None when edge_auth off) and trigger a paid Gemini run (main.py:1817 reads GEMINI_API_KEY; issue #340 documents re-billing) and read /api/videos responses. The UI is served same-origin (main.py:240,255 StaticFiles mounts), so wildcard CORS is not needed for the primary UX. Mitigated in current Chrome by Local Network Access prompts, NOT in Firefox.",  
  "why_it_matters": "Cost-leak plus local-data exposure reachable in normal use: a user browsing any hostile page while the appliance runs can have their Gemini key billed and their video index read. It silently defeats the P0c DNS-rebinding hardening the repo already invested in.",  
  "proposed_fix": "Change the default to same-origin/loopback (e.g. `[ ]` plus skipping the middleware, or an explicit localhost dev-origin list) and keep RALLY_CORS_ALLOW_ORIGINS as the widening knob; add a startup exposure/appliance warning when the wildcard is active. Add a regression test that an evil-origin preflight is rejected by default.",  
  "effort": "S",  
  "tracked_in": ""  
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator, C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field; non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup', C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).

VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training,

product/strategy/licensing, real cloud spend, counsel ToS/DPA?
HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you personally saw the defect/debt in current code; if you cannot reproduce the claim from the code, say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T02:58:15.270Z)

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
aa915a7 deploy/gcp: centralize project + bucket names in one sourced buckets.env (#410)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)

3. user (2026-07-02T02:58:15.529Z)

```
95         ``request.app.state`` ? eliminating the Optional module-level globals that forced
96         ``backend.main`` and ``backend.api.job_worker`` into mypy quarantine.
97
98     Also sets the module-level ``global_config`` / ``db_instance`` for transitional
99     compatibility (helpers and tests still reference them). Route handlers use
100     ``request.app.state`` exclusively."""
101     global global_config, db_instance
102     global_config = config
103     db_instance = db
104     app.state.config = config
105     app.state.db = db
106     return app
107
108
109     def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
110         """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
111         separated),
112         default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
113         ``backend.main``
114         does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
115         its origin."""
116         src = os.environ if env is None else env
117         raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
118         origins = [o.strip() for o in raw.split(",") if o.strip()]
119         return origins or ["*"]
120
121     # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
122     browsers per the
123     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
124     credentials stay
125     # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
126     deploy can
127     # lock it to its origin; the middleware is fixed at startup.
128     app.add_middleware(
129         CORSMiddleware,
130         allow_origins=_cors_origins_from_env(),
131         allow_credentials=False,
132         allow_methods=["*"],
133         allow_headers=["*"],
134     )
135
136     def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
137         """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
138         malicious
139         web page can point a domain it controls at 127.0.0.1 and script requests to the
140         local server;
141         the browser sends ``Host: attacker.com``, which this rejects (the server only
```

```

answers loopback
136     Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the
default keeps
137     Starlette's TestClient host ``testserver`` so the in-process suite isn't broken.
Read from the
138     ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
routable/edge
139     deploy sets its public host (or ``*`` when fronted by an auth gate that validates
the host)."""
140     src = os.environ if env is None else env
141     raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
142     hosts = [h.strip() for h in raw.split(",") if h.strip()]
143     # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
TrustedHostMiddleware
144     # matches on host.split(":")[0], which mangles a bracketed IPv6 Host ("[::1]:port"),
so IPv6
145     # loopback is NOT supported by this guard ? access the appliance via
127.0.0.1/localhost.
146     return hosts or ["localhost", "127.0.0.1", "testserver"]
147
148
149     # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
honoured
150     # (a remote page can't read responses cross-origin AND its Host header is rejected
here). www_redirect
151     # off so an unexpected ``www.`` host is rejected rather than 307-redirected.
152     app.add_middleware(
153         TrustedHostMiddleware,
154         allowed_hosts=_allowed_hosts_from_env(),
155         www_redirect=False,
156     )
157
158
159     def _resolve_bind_host(
160         config: Optional[AppConfig] = None, cli_host: Optional[str] = None
161     ) -> str:
162         """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
163         when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS
an auth gate
164         in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a
routable
165         interface with no auth gate is unsafe, hence the localhost default; widen it
consciously."""
166         if cli_host and cli_host.strip():
167             return cli_host.strip()
168         env = os.environ.get("RALLY_BIND_HOST")
169         if env and env.strip():
170             return env.strip()
171         sec = getattr(config, "security", None)
172         if getattr(sec, "edge_auth_enabled", False):
173             return "0.0.0.0" # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
174             return "127.0.0.1"
175
176
177     # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
01) ---
178
179
180     def _port_is_free(host: str, port: int) -> bool:
181         """True if (host, port) can be bound right now (best-effort; a TOCTOU race is
acceptable ?
182         uvicorn surfaces a real bind failure loudly)."""
183         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:

```

```

184         try:
185             s.bind((host, port))
186             return True
187         except OSError:
188             return False
189
190
191     def _find_free_port(host: str) -> int:
192         """An OS-assigned free port on host."""
193         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
194             s.bind((host, 0))
195             return int(s.getsockname()[1])
196
197
198     def _resolve_app_port(host: str, port: int) -> int:
199         """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
200         return port if _port_is_free(host, port) else _find_free_port(host)
201
202
203     def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
204         """Poll `health_url` until the server is up, then open `url` in the user's default
browser.
205         Returns True if a browser open was attempted. No-op (returns False) when
RALLY_NO_BROWSER is set
206         (tests / headless) or the server never becomes healthy (don't open a dead page).
Never raises."""
207         if os.environ.get("RALLY_NO_BROWSER"):
208             return False
209         import time as _time
210         import urllib.request
211
212         deadline = _time.monotonic() + timeout
213         while _time.monotonic() < deadline:
214             try:
215                 with urllib.request.urlopen(health_url, timeout=2) as r:
216                     if getattr(r, "status", 200) == 200:
217                         break
218             except Exception:
219                 _time.sleep(0.3)
220         else:
221             return False
222         try:
223             webbrowser.open(url)
224         except Exception:
225             pass
226         return True
227
228
229     # Serves static frontend files directly (now under api/ per approved structure).
230     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
231     # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
232     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
233     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
234     try:
235         os.makedirs(_STATIC_DIR, exist_ok=True)
236     except OSError:
237         # The dir ships with the package (a no-op create); guard against a read-only install
238         # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
PermissionError.
239         pass
240     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
241

```

```

242     # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
snapshot
243     # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
if it's
244     # absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
no SPA change
245     # is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets
is enough
246     # (no client-side path routing ? no history-fallback catch-all, which would also be a
route-order hazard).
247     _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
248     _UI_DIR = (
249         _BUNDLED_UI_DIR
250         if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
251         else _STATIC_DIR
252     )
253     _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
254     if os.path.isdir(_UI_ASSETS_DIR):
255         app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
256
257
258     @app.get("/", response_class=HTMLResponse)
259     def serve_index():
260         index_path = os.path.join(_UI_DIR, "index.html")
261         if os.path.exists(index_path):
262             with open(index_path, "r", encoding="utf-8") as f:
263                 return f.read()
264         return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"

```

4. user (2026-07-02T02:58:25.518Z)

```

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)
7ea6ea9 P3: /api/capabilities reports usable storage backends (#408)
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
759a67f feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)
5a5c4fa feat(api): P8a ? GET /api/source + POST /api/annotations (annotate-in-UI backend) (#405)
959c630 fix(#397): write per-video report to the resolved local path, not the raw URI (#400)
b6214e5 chore: ruff format the whole repo (259 files) (#387)
cd5b3d2 feat(01/P11): Gemini remote-file GC ? startup orphan sweep (#380)
9cd7598 fix(tests): resolve remaining test signature mismatches from app-factory refactor (#371)
de09ec8 refactor(main): app-factory + typed-config migration (D2/D3/P2/P3/P5/P7) (#370)
b25fdb8 docs: explain lazy cv2 import in --debug-clip path (Q8) (#361)
f523d3a refactor: enable isort (I) in ruff, auto-fix 143 files (Q6) (#360)
d92cc10 feat(serving): make the cloud compute-picker path work + provable (provenance +
skip_ai_handoff forward) (#341)
b6214e5 chore: ruff format the whole repo (259 files) (#387)
0c0e064 feat(serving): exposure-safety boot posture ? fail fast on a half-configured exposed
engine (default-OFF) (#294)
aac7633 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
(#290)

```

5. user (2026-07-02T02:58:26.076Z)

```

100         except KeyError as exc: # kid not in the JWKS
101             raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
102     except (
103         Exception
104     ) as exc: # jwt.* errors: bad sig / expired / wrong aud|iss / malformed
105         raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc
106
107     user = claims.get("email") or claims.get("sub")
108     if not user:
109         raise EdgeAuthError("verified token carries no email/sub claim")

```

```

110         return str(user)
111
112
113     def resolve_tenant(
114         headers: Mapping[str, str],
115         config: Any,
116         *,
117         jwks_fetcher: Optional[JwksFetcher] = None,
118     ) -> Optional[str]:
119         """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.
120
121         DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-
122         user; unchanged).
123         Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403)
124         on any miss.
125         ``jwks_fetcher`` is injectable for offline tests.
126         """
127         sec = getattr(config, "security", None)
128         if not sec or not getattr(sec, "edge_auth_enabled", False):
129             return None # DEFAULT-OFF ? no tenant identity, byte-identical to today
130
131         # Strip so a whitespace-only value counts as unset ? matching the boot-time exposure
132         # check in
133         # config/startup.py (EXPOSED_EDGE_AUTH_INCOMPLETE); one semantics for "configured",
134         # both places.
135         team_domain = (getattr(sec, "cf_team_domain", "") or "").strip()
136         aud = (getattr(sec, "cf_access_aud", "") or "").strip()
137         if not team_domain or not aud:
138             raise EdgeAuthError(
139                 "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not
140                 configured"
141             )
142
143         # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but
144         # support a plain
145         # dict too (tests) by trying both casings.
146         token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
147         if not token:
148             raise EdgeAuthError(
149                 f"missing {CF_ACCESS_HEADER} header (request did not pass Cloudflare
150                 Access)"
151             )
152
153         jwks = _get_jwks(team_domain, jwks_fetcher)
154         return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)
155
156

```

6. user (2026-07-02T02:58:44.991Z)

```

461:         "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
670:     Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is validated
671:     as a bare
733:     Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a local
734:     file from
1355:@app.post("/api/process", status_code=202)
1598:     Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip + concat +
1599:     watermark)
1817:         api_key = os.environ.get("GEMINI_API_KEY")

```

7. user (2026-07-02T02:58:50.687Z)

```

1355     @app.post("/api/process", status_code=202)
1356     def process_video(req: ProcessRequest, request: Request):
1357         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
1358

```

```

1359     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
1360     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
1361     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
1362     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
1363     ""
1364     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
1365     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
1366     # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
1367     try:
1368         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
1369     except edge_auth.EdgeAuthError as e:
1370         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
1371
1372     allowed_root = getattr(
1373         getattr(request.app.state.config, "security", None), "allowed_video_root", None
1374     )
1375     try:
1376         validate_input_path(req.video_path, allowed_root=allowed_root)
1377         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
1378         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
1379         input_ref = storage.resolve_input_ref(
1380             req.video_path, getattr(request.app.state.config, "storage", None)
1381         )
1382     except ValueError as e:
1383         raise HTTPException(status_code=400, detail=str(e)) from e
1384     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
1385     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
1386     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
1387     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
1388     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
1389     # rejects a non-local URI as out-of-root.
1390     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1391         raise HTTPException(
1392             status_code=404, detail=f"Video file not found at '{req.video_path}'"
1393         )
1394     # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
1395     # if this box can't hold that copy. Best-effort ? an unstatable remote
(network/SDK) or an
1396     # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
1397     # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
1398     if input_ref.backend != "local":
1399         need = _remote_input_size_bytes(
1400             input_ref, getattr(request.app.state.config, "storage", None)
1401         )
1402         require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
1403     if req.segmenter and not segmenter_registry.get(req.segmenter):
1404         available = list(segmenter_registry.list_available().keys())
1405         raise HTTPException(
1406             status_code=400,
1407             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
1408         )
1409

```

```

1410     job_id = uuid.uuid4().hex
1411     if not request.app.state.db.create_job(
1412         job_id,
1413         req.video_path,
1414         req.segmenter,
1415         req.player_pool,
1416         req.max_frames,
1417         owner_id=owner_id,
1418         locale=req.locale,
1419         compute=req.compute,
1420     ):
1421         raise HTTPException(status_code=500, detail="Failed to persist job record.")
1422     _get_executor().submit(
1423         _drain_due_jobs, request.app.state.config, request.app.state.db
1424     )
1425     return JSONResponse(
1426         status_code=202,
1427         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
1428     )
1429

```

8. user (2026-07-02T02:58:51.568Z)

```

1800     fail-fast wiring + the default-OFF no-op are unit-testable.""
1801     try:
1802         enforce_startup_checks(config, logger=logger, include_exposure=True)
1803     except StartupCheckError as e:
1804         logger.error("[STARTUP] refusing to start: %s", e)
1805         sys.exit(1)
1806
1807
1808     def _maybe_sweep_gemini_orphans(config) -> None:
1809         """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous
1810         crashed run.
1811         A no-op unless the configured provider is Gemini AND its API key is present. Never
1812         raises ?
1813         a failed sweep is logged and swallowed. The Gemini client is built with an HTTP
1814         timeout so
1815         a hung connection can't stall startup indefinitely. Extracted for unit tests.""
1816         try:
1817             provider_name = getattr(config, "ai_provider", "")
1818             if provider_name != "gemini":
1819                 return
1820             api_key = os.environ.get("GEMINI_API_KEY")
1821             if not api_key:
1822                 return
1823             provider = provider_registry.get(
1824                 "gemini", api_key=api_key, model_name=config.ai_model
1825             )
1826             deleted = provider.cleanup_orphaned_files()
1827             if deleted:
1828                 logger.info(
1829                     "[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted
1830                 )
1831         except Exception as e:
1832             logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
1833
1834
1835     def main():
1836         parser = argparse.ArgumentParser(
1837             description="Sports Highlight Indexer Orchestrator"
1838         )
1839         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
1840         parser.add_argument(
1841             "--output-dir",

```


1839 default="output",

9. user (2026-07-02T03:00:39.150Z)

Structured output provided successfully